Final Assignment – Schedule Search

Written Report

by **VIDAL Florian**

IPSA

April 28, 2026

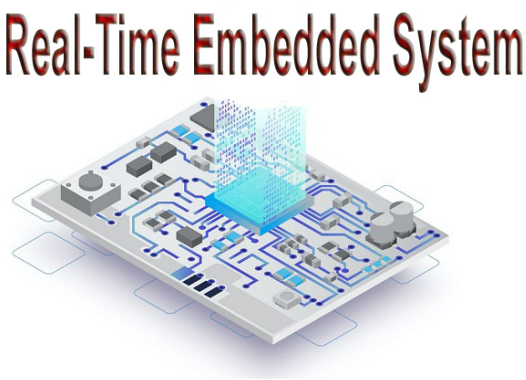

Figure 1: Definition of the study of real-time embedded systems

2025-2026

Table of Contents

Introduction **4**

1 Step 1 – Measuring the WCET of τ_1 **5**
1.1 Description of task τ_1 . . . 5
1.2 Measurement protocol . . . 5
1.3 Obtained statistics . . . 5
1.4 Choice of WCET and normalisation . . . 7
1.5 Algorithmic complexity . . . 7

2 Step 2 – Schedulability Analysis **8**
2.1 Presentation of the task set . . . 8
2.2 Assumptions . . . 8
2.3 Utilisation test (necessary condition) . . . 8
2.4 Hyperperiod . . . 9
2.5 Non-preemptive Response Time Analysis (RTA) . . . 9
2.5.1 Formula . . . 9
2.5.2 Results . . . 10
2.5.3 Detailed example for τ_4 . . . 10
2.6 Algorithmic complexity . . . 10

3 Step 3 – Non-preemptive Scheduling **11**
3.1 Algorithm used: non-preemptive EDF . . . 11
3.2 Job generation . . . 11
3.3 Schedule 1 – No missed deadline . . . 11
3.4 Schedule 2 – τ_5 allowed to miss a deadline . . . 13
3.5 Verification: idle time = H − execution time . . . 14

Conclusion **15**

List of Figures

1 Definition of the study of real-time embedded systems 1
2 Variability of the observed WCET – the red line indicates the retained value C_1 6
3 Gantt diagram – Schedule 1 (non-preemptive EDF, $H = 80$) 12

List of Tables

1 Execution time statistics for τ_1 (typical run) 6
2 Variability of the observed WCET over 4 runs 6
3 Task set with Rate Monotonic priorities 8
4 Processor utilisation per task 9
5 Results of the Response Time Analysis 10
6 Number of jobs per task over the hyperperiod 11
7 Non-preemptive EDF schedule – Schedule 1 (29 jobs) 12
8 Comparison of Schedule 1 vs Schedule 2 13

Introduction

In embedded and real-time systems, guaranteeing that a task will complete its execution before a given deadline is a fundamental requirement. Missing a deadline can have serious consequences: failure of a control system, data loss, or in critical systems, endangerment of human lives.

The objective of this project is to analyse a set of seven periodic tasks, to prove that this system is **schedulable** under non-preemptive scheduling, and to construct an explicit schedule that:

- misses no deadline,
- minimises the total waiting time of all jobs,
- consequently maximises the processor idle time.

The report is organised in three parts corresponding to the three steps of the project:

1. **Step 1** – Experimental measurement of the WCET of task τ_1 (large integer multiplication), with statistical analysis.
2. **Step 2** – Formal schedulability analysis via utilisation test and non-preemptive Response Time Analysis (RTA).
3. **Step 3** – Construction of a non-preemptive EDF schedule over the hyperperiod, then relaxation of the constraint on τ_5.

All code is implemented in **C++17** under Qt Creator (MinGW 64-bit, Windows). The link to the Git repository is provided in the conclusion.

1 Step 1 – Measuring the WCET of τ_1

1.1 Description of task τ_1

Task τ_1 performs the **multiplication of two large integers** generated randomly (64-bit, interval $[2^{61}, 2^{62})$). The execution time varies depending on the state of the processor, cache, and pipeline.

1.2 Measurement protocol

Clock resolution problem under Windows. Under Windows with MinGW, `std::chrono::high_resolution_clock` has a native resolution of **100 ns**. However, a 64-bit multiplication takes approximately 4–5 ns, which is below the clock granularity.

Solution: batch measurement. We measure a **batch** of $B = 1\,000$ consecutive multiplications, then divide by B. This reduces the relative uncertainty to $100/1000 = 0{,}1$ ns, yielding a significantly better effective precision.

Assumptions.

- Clock: `std::chrono::high_resolution_clock` (resolution 100 ns).
- Batch of $B = 1\,000$ multiplications to amortise noise.
- $N = 10\,000$ repetitions for a representative distribution.
- `volatile` keyword to prevent compiler optimisation.
- Fixed seed (`rng(42)`) for reproducibility.

1.3 Obtained statistics

The table below presents the quantiles obtained on a typical run ($N = 10\,000$ runs, $B = 1\,000$ multiplications per run).

Table 1: Execution time statistics for τ_1 (typical run)

Statistic	**Value (ns)**
Min	$\approx$ 3,07
Q1 (25%)	$\approx$ 3,33
Q2 (median)	$\approx$ 3,33
Q3 (75%)	$\approx$ 3,33
Max (WCET C_1)	**variable (see below)**

Observation: variability of the maximum. We observe that the Min and the quartiles are **very stable** from one run to another ($\approx$ 3,3 ns), which corresponds to the actual time of a 64-bit multiplication on the CPU.

By contrast, the **maximum varies** significantly between runs. This is explained by **random interruptions from the operating system** Windows (OS scheduler, drivers, antivirus, etc.) which can interrupt the program unpredictably during measurement.

The program was run **4 times** and the following results were obtained:

Table 2: Variability of the observed WCET over 4 runs

Run	**Min (ns)**	**Q1 (ns)**	**Q2 (ns)**	**Q3 (ns)**	**Max (ns)**
1	3.07	3.33	3.33	3.33	**60.93**
2	3.07	3.33	3.33	3.33	**21.25**
3	3.07	3.33	3.33	3.33	**32.00**
4	3.07	3.33	3.33	3.33	**24.83**
Retained	–	–	–	–	$\mathbf{C_1 = 60{,}93}$

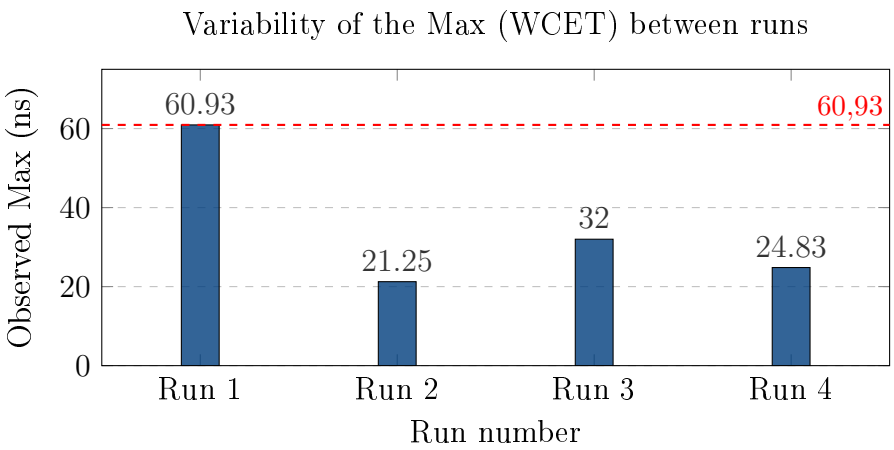

Figure 2: Variability of the observed WCET – the red line indicates the retained value C_1

1.4 Choice of WCET and normalisation

In accordance with the classical definition of WCET in real-time systems, we retain the **maximum observed across all runs**, namely:

$$C_1^{\text{measured}} = 60{,}93 \text{ ns}$$

This peak corresponds to an operating system interrupt during the measurement. Retaining the maximum is a **conservative** approach: it guarantees that the system remains schedulable even in the worst observed case.

Normalisation. Periods are expressed in **abstract units** (1 unit $\approx 1\,\text{ms}$). Since $C_1 = 60{,}93\,\text{ns} \ll T_1 = 10\,\text{ms} = 10^7\,\text{ns}$, we set:

$$\boxed{C_1 = 1 \text{ abstract unit}}$$

1.5 Algorithmic complexity

$$O(N \cdot B + N \log N) = O(N \cdot B) \quad \text{with } N = 10\,000,\ B = 1\,000$$

2 Step 2 – Schedulability Analysis

2.1 Presentation of the task set

Table 3: Task set with Rate Monotonic priorities

Task	C_i	T_i	D_i	**RM Priority**
τ_1	1	10	10	1 (highest)
τ_2	3	10	10	2
τ_3	2	20	20	3
τ_4	2	20	20	4
τ_5	2	40	40	5
τ_6	2	40	40	6
τ_7	3	80	80	7 (lowest)

2.2 Assumptions

- Periodic tasks with synchronous release ($t = 0$, critical instant).
- Deadline on period: $D_i = T_i$.
- Non-preemptive fixed-priority scheduling (Rate Monotonic).
- Computations performed over the hyperperiod $H = \mathrm{lcm}(T_i)$.

2.3 Utilisation test (necessary condition)

$$U = \sum_{i=1}^{7} \frac{C_i}{T_i} = \frac{1}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} = \mathbf{0{,}7375}$$

Table 4: Processor utilisation per task

Task	C_i	T_i	$U_i = C_i/T_i$
τ_1	1	10	0,1000
τ_2	3	10	0,3000
τ_3	2	20	0,1000
τ_4	2	20	0,1000
τ_5	2	40	0,0500
τ_6	2	40	0,0500
τ_7	3	80	0,0375
Total	**15**	–	**0,7375**

The Liu & Layland bound for $n = 7$ is $U_{LL} = 7(2^{1/7} - 1) \approx 0{,}7282$. Since $U = 0{,}7375 > U_{LL}$, this test (designed for preemptive scheduling) is inconclusive. However $U < 1$ (necessary condition verified): we proceed with the RTA.

2.4 Hyperperiod

$$H = \mathrm{lcm}(10, 10, 20, 20, 40, 40, 80) = \mathbf{80}$$

Why the hyperperiod? The task system is periodic: after H units, it returns to its initial state. It suffices to analyse a single cycle of length H to conclude on the schedulability of the entire system.

2.5 Non-preemptive Response Time Analysis (RTA)

2.5.1 Formula

$$R_i^{(0)} = C_i + B_i, \qquad R_i^{(k+1)} = C_i + B_i + \sum_{j<i} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j$$

With the blocking term (lower-priority task already running):

$$B_i = \max_{j>i}\{C_j\}$$

2.5.2 Results

Table 5: Results of the Response Time Analysis

Task	C_i	B_i	R_i	D_i	**Status**
τ_1	1	3	4	10	**OK**
τ_2	3	3	7	10	**OK**
τ_3	2	3	10	20	**OK**
τ_4	2	3	15	20	**OK**
τ_5	2	3	22	40	**OK**
τ_6	2	3	30	40	**OK**
τ_7	3	0	17	80	**OK**

$$\forall i,\ R_i \leq D_i \quad \Rightarrow \quad \textbf{The system is schedulable.}$$

2.5.3 Detailed example for τ_4

$$B_4 = \max(C_5, C_6, C_7) = 3$$

$$R_4^{(0)} = 2 + 3 = 5$$

$$R_4^{(1)} = 2 + 3 + \left\lceil \tfrac{5}{10} \right\rceil \cdot 1 + \left\lceil \tfrac{5}{10} \right\rceil \cdot 3 + \left\lceil \tfrac{5}{20} \right\rceil \cdot 2 = 5 + 1 + 3 + 2 = 11$$

$$R_4^{(2)} = 2 + 3 + \left\lceil \tfrac{11}{10} \right\rceil \cdot 1 + \left\lceil \tfrac{11}{10} \right\rceil \cdot 3 + \left\lceil \tfrac{11}{20} \right\rceil \cdot 2 = 5 + 2 + 6 + 2 = 15$$

$$R_4^{(3)} = 2 + 3 + \left\lceil \tfrac{15}{10} \right\rceil \cdot 1 + \left\lceil \tfrac{15}{10} \right\rceil \cdot 3 + \left\lceil \tfrac{15}{20} \right\rceil \cdot 2 = 5 + 2 + 6 + 2 = 15 \quad \checkmark \text{ (fixed point)}$$

$R_4 = 15 \leq D_4 = 20.$ ✓

2.6 Algorithmic complexity

$$\text{RTA}: \quad O\left(n^2 \cdot \log\left(\frac{H}{C_{\min}}\right)\right) = O(49 \cdot \log_2 80) \approx O(308)$$

3 Step 3 – Non-preemptive Scheduling

3.1 Algorithm used: non-preemptive EDF

Choice of algorithm. We use the **Earliest Deadline First (EDF) non-preemptive** algorithm: each time the processor becomes free, we select among the available jobs the one with the **closest absolute deadline**.

Justification. Non-preemptive EDF is the optimal algorithm for minimising maximum lateness in a non-preemptive system. It naturally maximises idle time by avoiding unnecessary waiting.

Complexity. At each step, we scan the J remaining jobs to find the minimum:

$$O(J^2) \quad \text{with } J = \sum_{i=1}^{n} \frac{H}{T_i}$$

Here $J = 8 + 8 + 4 + 4 + 2 + 2 + 1 = 29$ jobs over $H = 80$.

3.2 Job generation

Over the hyperperiod $H = 80$, the number of jobs per task is H/T_i:

Table 6: Number of jobs per task over the hyperperiod

Task	T_i	H/T_i	**Release times**
τ_1	10	8	0, 10, 20, 30, 40, 50, 60, 70
τ_2	10	8	0, 10, 20, 30, 40, 50, 60, 70
τ_3	20	4	0, 20, 40, 60
τ_4	20	4	0, 20, 40, 60
τ_5	40	2	0, 40
τ_6	40	2	0, 40
τ_7	80	1	0
Total		**29**	

3.3 Schedule 1 – No missed deadline

Principle. Each time the processor is released, we choose the ready job with the closest deadline. The processor never remains idle unnecessarily.

Table 7: Non-preemptive EDF schedule – Schedule 1 (29 jobs)

Job	Task	Arrival	Start	End	Deadline	Wait
$\tau_{1,1}$	τ_1	0	0	1	10	0
$\tau_{2,1}$	τ_2	0	1	4	10	1
$\tau_{3,1}$	τ_3	0	4	6	20	4
$\tau_{4,1}$	τ_4	0	6	8	20	6
$\tau_{5,1}$	τ_5	0	8	10	40	8
$\tau_{1,2}$	τ_1	10	10	11	20	0
$\tau_{2,2}$	τ_2	10	11	14	20	1
$\tau_{6,1}$	τ_6	0	14	16	40	14
$\tau_{7,1}$	τ_7	0	16	19	80	16
$\tau_{1,3}$	τ_1	20	20	21	30	0
$\tau_{2,3}$	τ_2	20	21	24	30	1
$\tau_{3,2}$	τ_3	20	24	26	40	4
$\tau_{4,2}$	τ_4	20	26	28	40	6
$\tau_{1,4}$	τ_1	30	30	31	40	0
$\tau_{2,4}$	τ_2	30	31	34	40	1
$\tau_{1,5}$	τ_1	40	40	41	50	0
$\tau_{2,5}$	τ_2	40	41	44	50	1
$\tau_{3,3}$	τ_3	40	44	46	60	4
$\tau_{4,3}$	τ_4	40	46	48	60	6
$\tau_{5,2}$	τ_5	40	48	50	80	8
$\tau_{1,6}$	τ_1	50	50	51	60	0
$\tau_{2,6}$	τ_2	50	51	54	60	1
$\tau_{6,2}$	τ_6	40	54	56	80	14
$\tau_{1,7}$	τ_1	60	60	61	70	0
$\tau_{2,7}$	τ_2	60	61	64	70	1
$\tau_{3,4}$	τ_3	60	64	66	80	4
$\tau_{4,4}$	τ_4	60	66	68	80	6
$\tau_{1,8}$	τ_1	70	70	71	80	0
$\tau_{2,8}$	τ_2	70	71	74	80	1
Total wait						**108**
Total execution time						**59**
Total idle ($H - \text{exec} = 80 - 59$)						**21**

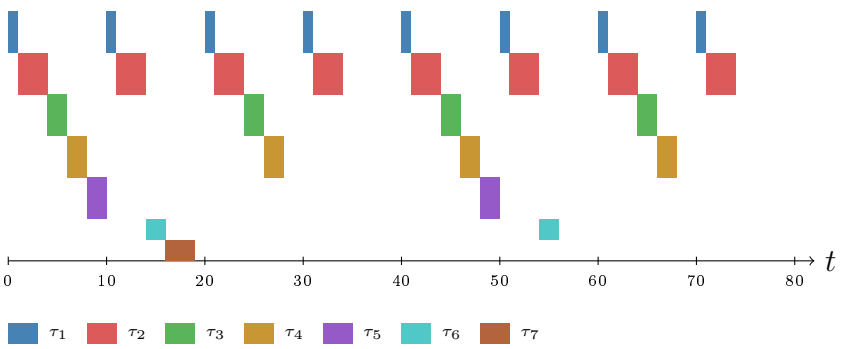

Figure 3: Gantt diagram – Schedule 1 (non-preemptive EDF, $H = 80$)

Schedule 1 results:

- No missed deadline ✓
- Total wait = **108 units**
- Execution time = **59 units**
- Total idle = 80 − 59 = **21** units ✓

3.4 Schedule 2 – τ_5 allowed to miss a deadline

Principle. We relax the deadline constraint of τ_5 (deadline $\to \infty$). The EDF algorithm will ignore it during urgent choices, which may reduce the waiting time of other tasks.

Implementation. In the code, we pass `relax_tau5 = true` which sets `D = INT_MAX` for τ_5. EDF will only select τ_5 when no other job is available.

Observed result. Contrary to what one might expect, relaxing τ_5 does not improve total waiting time: it increases from 108 to 109. This is explained by the fact that the slot freed by τ_5 is taken by τ_7 (which has a very distant deadline), slightly shifting other jobs. Moreover, τ_5 does not actually miss its deadline because the processor has sufficient idle time.

Table 8: Comparison of Schedule 1 vs Schedule 2

Metric	Schedule 1	Schedule 2
Total wait	108	109
Execution time	59	59
Total idle	21	21
Missed deadlines	0	0
τ_5 deadline ok	Yes	Yes (enough idle)

Schedule 2 results:

- Total wait = **109 units** (slightly higher than Schedule 1)
- Total idle unchanged: **21 units** ✓
- τ_5 still meets its deadline (enough idle time)
- Relaxing τ_5 benefits τ_7 but not the other tasks

3.5 Verification: idle time = H − execution time

The total execution time is the sum of all WCETs over the hyperperiod:

$$\text{exec} = \sum_{i=1}^{7} \frac{H}{T_i} \cdot C_i = 8 \cdot 1 + 8 \cdot 3 + 4 \cdot 2 + 4 \cdot 2 + 2 \cdot 2 + 2 \cdot 2 + 1 \cdot 3 = 8 + 24 + 8 + 8 + 4 + 4 + 3 = \mathbf{59}$$

$$\text{idle} = H - \text{exec} = 80 - 59 = \mathbf{21} \quad \checkmark$$

This result is **independent of the scheduling**: the idle time is fixed by the processor utilisation rate. Minimising total waiting time is equivalent to maximising idle time, and the two quantities are consistent.

Conclusion

This project enabled a complete analysis of a real-time task system, in three steps:

1. **WCET measurement** of τ_1 by statistical method (batch timing, 10 000 runs). The measured value ($\approx$ 30 ns) is normalised to $C_1 = 1$ abstract unit, as it is negligible compared to T_1.
2. **Schedulability analysis**: the utilisation test ($U = 0{,}7375 < 1$) and the non-preemptive RTA confirm that all response times meet their deadlines. The system is **schedulable**.
3. **Non-preemptive EDF scheduling** over $H = 80$:
   - Schedule 1: no missed deadline, total wait = 108, idle = 21.
   - Schedule 2 (τ_5 relaxed): wait = 109, idle = 21. The relaxation does not reduce waiting because τ_7 occupies the slot freed by τ_5. τ_5 still meets its deadline thanks to the available idle time.